RepoGym

Product Requirements Document (MVP)

Field	Value
Version	1.0 (MVP)
Date	November 4, 2026
Owner	Sarthak and team
Status	Draft for mentor review

1. Overview

RepoGym is a training platform that turns any public GitHub repository into an interactive learning session. The student pastes a repo URL, and RepoGym maps the code, shows how its files connect in an interactive graph, and asks a sequence of questions about how the code works. The AI grades each answer against the actual code and cites the file and line behind every point.

The goal is to help students and developers understand code they didn't write or can't explain, which is increasingly common as AI generates more of it.

2. Problem Statement

Developers can now produce code with AI faster than they can understand it. Students, engineers onboarding onto unfamiliar codebases, and companies training new hires all end up working with code they can't fully explain.

Our team's research, including interviews with 11 people, showed this pattern clearly:

- During a Hacktoberfest hackathon, students built working prototypes but could not explain how they worked in evaluations.
- Students could not explain their college assessment projects during evaluations, even though they had built them.

The gap is understanding, not building. Existing tools either explain code to the reader (documentation) or generate code for the developer. None test whether the developer understands the code.

3. Goals and Success Metrics

Goals

- Let a user go from a GitHub URL to a graded learning session in under 2 minutes for a typical repo.
- Show how the files in a repo connect, through an interactive graph.
- Ask a sequence of questions, where each question adapts to the answer before it.
- Grade answers against the actual code, with citations the user can verify.
- Save each repo's progress, so users can return and resume.

Success metrics (MVP demo)

Metric	Target
Time from URL to first question	Under 60 seconds for repos under 150 files
Citation accuracy (cited lines exist and support the grade)	95%+ on a test set of 10 repos
Users who finish a full session	60% of testers
Testers who rate feedback as helpful (thumbs up)	70%+
Repeat visits to a saved repo	At least 3 of 10 testers return

4. Target Users

Segment	Description	Priority
Students	First to final year students who build hackathon and college projects and must explain them confidently	Primary (MVP)
Engineers onboarding	Developers joining a new codebase who need to understand its architecture quickly	Secondary (MVP)
Companies	Teams training new hires on internal codebases	Future

5. Scope

In scope (MVP)

- Public GitHub repositories only
- JavaScript, TypeScript, and Python source files
- One flow type: request tracing (for example, what happens on login)
- Sequential questions in a session (default: 3 per session)
- Interactive dependency graph of imports and exports
- Grading with score, verdict, covered and missed points, and citations
- Follow-up question per grade
- Thumbs up or down on feedback
- User accounts with saved repos, sessions, and attempts
- Dashboard with repo list, session progress, and performance overview

Out of scope (MVP)

- Private repositories
- Running or executing student code
- Bug hunting, output prediction, and commit investigation challenges
- The AI interviewer
- Adaptive difficulty beyond topic targeting
- Team, classroom, or company accounts
- Payments and subscriptions
- Languages other than JavaScript, TypeScript, and Python
- Mobile app

6. User Stories

#	As a...	I want to...	So that...
U1	Student	Paste a GitHub URL	I can start learning from a real project
U2	Student	See how files connect in a graph	I understand the structure at a glance
U3	Student	Click a file in the graph to read its code	I can see the code behind a node
U4	Student	Answer a question in my own words	I practice explaining the code

#	As a...	I want to...	So that...
U5	Student	Get a grade with citations	I know exactly what I got right and wrong
U6	Student	Receive a follow-up question	I work on what I missed
U7	Student	Resume a session after closing the tab	I don't lose progress
U8	Student	See my past sessions and scores	I can track my improvement
U9	Student	Re-open a repo I analyzed before	I don't wait for it to load again
U10	Student	Mark feedback as helpful or not	The product improves over time
U11	Student	See which topics I miss most	I know what to study next
U12	Student	Be told when a repo has changed	I can practice the new code

7. Features and Requirements

F1: Repository input

- Input: a GitHub URL, with or without https://, .git, or a trailing path.
- Validation: reject non-GitHub and malformed URLs with a clear message.
- Private or missing repos (404 or 403): show that RepoGym supports public repos only.
- Empty or unsupported repos: show that no supported code files were found.
- Saved repos: if the user has analyzed this repo before, show the saved analysis and sessions instead of re-fetching.

F2: Repository analysis

- Fetch the file tree from the GitHub API for the default branch.
- Filter files. Include .js, .jsx, .ts, .tsx, and .py. Exclude node_modules, dist, build, vendor, .git, lockfiles, and files over 20 KB. Cap at 150 files.
- Fetch file contents in batches of 10 concurrent requests.
- Build the file map as { path: content }.
- Build the import graph (see F3).
- Detect routes (Express app.get/post, Next.js app/api route files, FastAPI decorators) to populate the list of flows.
- Store the analysis keyed by repo_id and commit_sha.
- Show status updates: Fetching files, Mapping imports, Finding flows, Ready.
- Truncated trees: if GitHub reports the tree as truncated, show a notice that only part of the repo was analyzed.

F3: Dependency graph

Data:

- Nodes: one per analyzed file, holding the file path, language, list of exports, and number of imports.
- Edges: one per resolved relative import, from the importing file to the imported file.

• Unresolved imports (external packages, dynamic imports) are not drawn as edges but are listed in the file's details.

Export detection:

• ES modules: export function, export const, export class, export default.

• CommonJS: module.exports = ..., exports.name = ...

Display:

• Interactive canvas built with React Flow.

• Automatic left-to-right layout using dagre, following import direction.

• Zoom, pan, fit-to-view, and minimap controls.

• Drag to reposition nodes (positions are not saved in the MVP).

• Each node shows the file path and its exports.

Interactions:

• Click a node: opens a side panel with the file's code, with line numbers.

• Click an edge: shows the import statement that creates it.

• Question highlight: when a question is active, the files it cites are highlighted in yellow.

• Flow filter toggle: shows only the files in the current flow, for large repos.

Limits: if the repo has more than 80 files, the full graph is hidden by default and the flow filter is on.

F4: Challenge generation

• Input to the AI: the flow's files with line numbers, the graph edges for those files, and the task.

• Flow selection: the system chooses a flow from the routes found in F2. The student can pick from the detected flows.

• Output: a question, a list of expected facts, and for each fact, the file and line range that supports it.

• Validation: every cited file and line must exist in the file map. Invalid challenges are regenerated once; then the student sees a message to try another flow.

F5: Sequential questions (sessions)

• Session start: the student starts a session on a saved analysis. Default length: 3 questions.

• Question order: each question is generated after the previous answer is graded.

• Context for the next question: a summary of past results (flow, verdict, missed points). The AI must not repeat a flow already asked in the session.

• Targeting: if the student missed a topic, the next question prefers that topic when the code supports it.

• Session end: the student sees a summary (see F9).

• Abandoned sessions: a session with no activity for 7 days is marked paused and can be resumed.

F6: Answer submission and grading

- Input: a free-text answer, up to 2,000 characters.
- Grading prompt includes: the question, expected facts, cited code with line numbers, and the student's answer.
- Output: score and maxScore; verdict (correct, partial, or incorrect); covered and missed fact IDs; feedback; citations; and followUpQuestion (null if the answer was correct).
- Validation (Zod schema): all fields present and typed; score between 0 and maxScore; every covered or missed ID matches an expected fact; every citation exists in the file map.
- Failure handling: retry once. If it fails again, show that the grade couldn't be generated, keep the answer saved, and allow a retry.

F7: Feedback and trust

- Each grade shows thumbs up and thumbs down.
- A thumbs down can include an optional short note.
- Feedback is stored with the attempt, so the team can review low-rated grades.

F8: Accounts and authentication

- Sign in with GitHub or email through Supabase Auth. Supabase issues the session, and the Next.js server checks it on each request.
- Each user's sessions, attempts, and feedback are private to them.
- Analyses (file maps, graphs) are shared across users, keyed by repo and commit.
- Sign out clears the session.

F9: Dashboard

My repositories:

- List of repos the user has added.
- For each repo: name, last visited date, analyzed commit, number of sessions, and progress of the latest session.
- Buttons: Resume and Start new session.
- Badge when a newer commit exists: Repo updated, re-analyze.

Repo page:

- Dependency graph (F3).
- Session list with date, score, and status (in progress, completed, paused).
- Start new session and Re-analyze buttons. Re-analyze creates a new analysis for the latest commit and keeps old sessions.

Session view:

- Question number out of the total (for example, Question 2 of 3).
- Question text, answer box, and graph with highlights.

- Previous answers and grades in this session, collapsed by default.

Session summary:

- Total score and percentage.
- Topics covered and missed.
- List of questions with verdicts.
- Review missed topics button, which starts a session targeting those topics.

Performance overview:

- Score trend across completed sessions (line chart).
- Topics with the most misses, grouped by topic tag (for example, auth, routing, database).
- One suggested focus area, based on the most-missed topic.

Feedback history: all grades the user received, with citations and thumbs status.

F10: Re-analysis and updates

- When a repo has a newer default-branch commit, the app shows a badge.
- Re-analysis creates a new analysis record. Existing sessions keep pointing to their original analysis.
- Users are never forced to re-analyze.

F11: Caching and reuse

- If an analysis exists for the same repo and commit, it is reused with no GitHub or AI calls.
- If a challenge exists for the same analysis and flow, it is reused.
- Caching makes repeat visits instant and protects the free AI quota.

8. User Flows

Flow A: First-time analysis and first session

- 1. User signs in.
- 2. User pastes a GitHub URL on the home page and clicks Analyze.
- 3. System validates the URL.
- 4. System fetches the file tree and file contents, showing status updates.
- 5. System builds the file map, import graph, and list of detected flows.
- 6. User lands on the repo page, which shows the graph and the detected flows.
- 7. User clicks Start session and picks a flow (or accepts the default).
- 8. System generates question 1 for that flow.
- 9. User reads the question; highlighted files appear on the graph; user writes an answer.
- 10. User submits. System grades the answer.
- 11. User sees the grade, citations (which highlight the cited files), and the follow-up question.

- 12. User clicks Next question. System generates question 2, using previous results.
- 13. Steps 9 to 12 repeat until the session ends.
- 14. User sees the session summary and can start a targeted session on missed topics.

Error paths:

- Invalid or private repo: show the F1 error and stay on the home page.
- No flows detected: show the graph with a message that no request flows were found; the graph remains explorable.
- AI generation fails: show a retry button and do not save an invalid question.

Flow B: Returning user resumes a session

- 1. User signs in and lands on the dashboard.
- 2. User sees My repositories with progress on each repo.
- 3. User clicks Resume on a repo with an in-progress session.
- 4. System loads the session, the current question, and previous attempts.
- 5. User continues from the current question.

Flow C: Returning user, repo has changed

- 1. User opens a saved repo and sees Repo updated, re-analyze.
- 2. User clicks Re-analyze.
- 3. System creates a new analysis for the latest commit.
- 4. Existing sessions are kept; new sessions use the new analysis.

Flow D: Explore the graph without a session

- 1. User opens a repo page.
- 2. User clicks a node to read its code.
- 3. User clicks edges to see import statements.
- 4. User toggles the flow filter to focus on one flow.

Flow E: Give feedback on a grade

- 1. User clicks thumbs up or thumbs down on a grade.
- 2. If thumbs down, a short optional text box appears.
- 3. System stores the feedback with the attempt.

Flow F: Review performance

- 1. User opens the dashboard performance tab.
- 2. User sees the score trend and most-missed topics.

- 3. User clicks the suggested focus area.
- 4. System starts a session targeting that topic.

9. Functional Requirements (API and Data)

API endpoints

Method	Endpoint	Purpose
POST	/api/repos	Validate URL, fetch and analyze repo, or return saved analysis
GET	/api/repos	List the user's repos with progress
GET	/api/repos/[id]	Repo details, graph, and session list
POST	/api/repos/[id]/reanalyze	Create a new analysis for the latest commit
GET	/api/analyses/[id]/files/[...path]	Return one file's content with line numbers
POST	/api/sessions	Start a session on an analysis with a chosen flow
GET	/api/sessions/[id]	Session state, current question, previous attempts
POST	/api/sessions/[id]/next	Generate the next question
POST	/api/questions/[id]/answers	Submit an answer; return grade and follow-up
POST	/api/attempts/[id]/feedback	Store thumbs up or down
GET	/api/dashboard	Performance overview and feedback history

All endpoints require a signed-in user.

System architecture

```
Browser (Next.js pages, React Flow graph)
  -> Next.js API routes (server code, hosted on Vercel)
       -> GitHub REST API and raw file URLs
       -> OpenRouter (Nemotron model)
       -> Supabase (PostgreSQL database and Auth)
```

- The browser never calls GitHub, OpenRouter, or the database directly. It only calls the app's own API routes.
- API routes hold all secrets (GitHub token, OpenRouter key, service role key) and make all external calls.
- Supabase Auth manages sign-in. Row Level Security protects user data inside the database.

Data model

```
users            id, email, github_username, created_at
repos            id, owner, name, default_branch, created_at
analyses         id, repo_id, commit_sha, file_map (jsonb), graph (jsonb),
                 flows (jsonb), truncated (bool), created_at
                 unique (repo_id, commit_sha)
sessions         id, user_id, analysis_id, flow_tag, status,
                 total_questions, current_index, created_at, updated_at
questions        id, session_id, analysis_id, flow_tag, position,
                 question, expected_facts (jsonb), created_at
attempts         id, question_id, user_id, answer, score, max_score, verdict,
                 covered (jsonb), missed (jsonb), feedback, citations (jsonb),
                 follow_up_question, created_at
feedback_flags   id, attempt_id, user_id, helpful (bool), note, created_at
```

- analyses is shared across users. Sessions and attempts are private.
- questions belong to a session, so each session has its own question sequence.

- Access control: Row Level Security in Supabase ensures users can only read or write their own sessions, attempts, and feedback. The browser uses only the public anon key together with the signed-in user's session.

10. Data Fetching and Processing

GitHub data

Step	Call	Purpose
1	GET /repos/{owner}/{repo}	Confirm the repo exists; get default_branch
2	GET /repos/{owner}/{repo}/commits/{branch}	Get the latest commit_sha (cache key)
3	GET /repos/{owner}/{repo}/git/trees/{branch}?recursive=1	Get the full file list in one call
4	GET raw.githubusercontent.com/{owner}/{repo}/{branch}/{path}	Download each file's content

- Use a GITHUB_TOKEN on the server to raise the rate limit from about 60 to 5,000 requests per hour.
- All GitHub and AI calls are made from Next.js server code (API routes), so tokens are never sent to the browser.
- Download files in batches of 10 in parallel.
- Skip files over 20 KB before downloading.
- If the tree response reports truncated: true, mark the analysis as truncated and show a notice.
- Read the X-RateLimit-Remaining header and show a friendly error if it runs out.

Import parsing

- Method: regular expressions over file contents for the MVP.
- Patterns: import ... from './x', require('./x'), and import('./x').
- Resolution: try the exact path, then .js, .ts, /index.js, and /index.ts.
- Skipped: external packages (no leading dot) and paths that do not resolve.

Route detection (for flows)

- Express: app.get('/path') and router.post('/path') calls.
- Next.js: files under app/api/ or pages/api/.
- FastAPI and Flask: @app.get, @app.post, @app.route.
- Each detected route becomes a candidate flow, with the route file and its direct imports as the flow's files.

AI calls

- Provider: OpenRouter, using the OpenAI-compatible SDK.
- Primary model: nvidia/nemotron-3-ultra-550b-a55b:free.
- Fallback model: configured in the same file (for example, Gemini Flash or Groq).

- Prompt budget: under 30,000 tokens (about 100 KB of code) per request. If over, drop the least relevant file first.
- Output: requested as JSON. The server extracts the JSON object from the reply and validates it with Zod.
- Timeouts: 30 seconds per call; one retry on invalid JSON or timeout.
- Logging: prompt size, latency, model, and validation result. Do not log file contents or answers in plain text.

11. Non-Functional Requirements

Area	Requirement
Performance	Analysis under 60 seconds for repos up to 150 files. Graph renders under 2 seconds for up to 80 nodes. Grading responds under 15 seconds. Dashboard loads under 2 seconds.
Reliability	If the AI fails, the student keeps their answer and can retry. If GitHub is rate-limited, show a retry time and keep the user's place. Saved data survives refresh and new sign-ins.
Security and privacy	API keys are stored as server-side environment variables only. Only public repos are processed. Answers and feedback are visible only to their owner. Users are told that code from public repos is sent to the model provider. Student answers are rendered as plain text, not HTML.
Cost control	Caching by commit SHA avoids repeat analysis and AI calls. A limit of 20 new sessions per user per day protects the free quota. Each analysis is capped at 150 files and 200 KB.
Accessibility	Keyboard navigation for the graph (arrow keys between nodes). Highlighted nodes use a border and label, not color alone. Text meets standard contrast ratios.
Compatibility	Latest Chrome, Edge, Firefox, and Safari on desktop. Responsive layout for sessions and the dashboard on mobile; the graph is optimized for desktop.

12. Tech Stack

Layer	Choice	Why
Framework	Next.js (App Router) with TypeScript	Pages and API routes in one project
Styling	Tailwind CSS	Fast, consistent UI
Graph	React Flow (@xyflow/react)	Interactive nodes, zoom, drag, clicks
Layout	dagre (@dagrejs/dagre)	Automatic left-to-right graph layout
Validation	Zod	Checks AI JSON output and API input
GitHub	GitHub REST API and raw content URLs	Fetch file tree and contents
AI gateway	OpenRouter with openai SDK	One key, OpenAI-compatible format
AI model	Nemotron 3 Ultra (free), with fallback	Strong code reasoning at no cost
Database	Supabase (PostgreSQL)	Relational data for users, repos, sessions, and attempts
Auth	Supabase Auth (GitHub and email)	Sign-in and per-user sessions
Security	Supabase Row Level Security	Users can only access their own rows
Hosting	Vercel	Native Next.js hosting, environment variables
Source control	GitHub	Standard workflow

Environment configuration (.env.local)

All keys are stored in a .env.local file in the project root, next to package.json. The file is never committed to GitHub. The same variables are added in Vercel's project settings for the deployed app. Only variables

prefixed with NEXT_PUBLIC_ are sent to the browser.

Variable	Purpose	Where it is used	Exposed to browser
OPENROUTER_API_KEY	Authenticates calls to the AI model through OpenRouter	Server (lib/llm.js)	No
GITHUB_TOKEN	Raises the GitHub rate limit; needed for reliable analysis	Server (lib/github.js)	No
NEXT_PUBLIC_SUPABASE_URL	Address of the Supabase project	Server and browser	Yes (safe)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public key that respects Row Level Security	Server and browser	Yes (safe)
SUPABASE_SERVICE_ROLE_KEY	Admin key that bypasses Row Level Security	Server only	No

- Obtain values from OpenRouter (Keys), GitHub (Developer settings, fine-grained read-only token for public repos), and Supabase (Project Settings, API).
- Confirm .env*.local is listed in .gitignore (create-next-app adds it by default).
- Restart the development server after any change, because Next.js reads the file at startup.
- If a key is ever exposed (screenshot, commit, or chat), revoke it in the provider's dashboard and create a new one.
- Optional later: NEXT_PUBLIC_APP_URL and a fallback model key (GEMINI_API_KEY or GROQ_API_KEY).

Build decision: the pitch deck planned to build on Replit. This PRD assumes GitHub and Vercel, which need no credits and are easier to hand off.

13. Error Handling Summary

Situation	What the user sees	System action
Invalid URL	Enter a valid GitHub repository URL	Block request
Private or missing repo	This repo is private or doesn't exist	No analysis saved
No supported files	No JavaScript, TypeScript, or Python files found	No analysis saved
GitHub rate limit	GitHub is busy; try again in X minutes	Keep user's place
Truncated repo tree	Only part of this repo was analyzed	Mark analysis truncated
No flows detected	Graph with a no-flows message	Graph still available
AI invalid JSON	Couldn't generate a question; retry?	Retry once, then show error
Citation check fails	Question regenerated silently	Retry once
Grading fails	Couldn't grade this answer; your answer is saved	Save answer; allow retry
Session already completed	Show summary	Block new answers

14. Analytics and Logging

Track these events, anonymized where possible:

- Repo analyzed (repo ID, file count, analysis duration).
- Session started and completed.

- Questions generated, with validation pass or fail.
- Grades returned, with verdict distribution.
- Thumbs up and down, with the grade's verdict.
- Errors by type (GitHub, AI, validation, database).

15. Milestones (Compressed Timeline)

The original plan allowed three weeks, but the team has about nine days before the November 12 demo. Scope is set to match.

Days	Milestone
1 to 2	Repo fetcher, file map, import graph, and graph data (no AI)
3	React Flow graph with dagre layout, clicks, and file viewer
4	One challenge generated from one flow, with citation check
5	Grading with Zod validation and feedback UI
6	Sequential questions in a session (3 per session)
7	Supabase: accounts, saved analyses, sessions, attempts
8	Dashboard: repo list, session progress, summary
9	Deploy, test on 3 to 4 repos, fix critical issues, prepare demo

Priority if time runs short: graph (Day 3), one graded question (Day 5), and saved sessions (Day 7). Cut the performance overview, re-analysis, and feedback history first.

16. Risks and Mitigations

Risk	Impact	Mitigation
Free AI model unreliable or rate-limited	Demo fails	Fallback model, caching, pre-generated demo repo
Wrong or invented grading	Trust lost	Citation check, thumbs down, visible sources
Import regex misses patterns	Incomplete graph	Log unresolved imports; document as a limitation
Large repos slow to analyze	Poor first impression	150-file cap; demo on small repos
Replit credits still pending	Build delayed	Build on GitHub and Vercel, which need no credits
Supabase or Vercel outage during demo	Demo fails	Check services before the demo; keep a recorded walkthrough as backup
Scope creep	Missed deadline	Feature cut list in Section 15
Privacy concerns with free model	User trust	Public repos only; clear disclosure

17. Open Questions

- Which two or three repos will we use for the demo, and do we have each one's expected answers ready?
- Is GitHub login required for the demo, or is email sign-in enough?
- Should the MVP support flows beyond request tracing (for example, function call chains)?
- How many testers can we recruit before the demo, and who reviews the feedback?

- Do mentors want to see the graph first or the grading first?

18. Future Scope

- Additional challenge types: bug hunting, output prediction, design decision explanations.
- Git commit investigation challenges.
- Private repository support through GitHub App authentication.
- Adaptive difficulty based on the live skill profile.
- Code execution in an isolated sandbox.
- Language support beyond JavaScript, TypeScript, and Python.
- Team and classroom accounts for educators and companies.
- The AI interviewer, asking questions about the student's own code in a conversational format.
- Saved graph layouts and annotations.